📕 ПРЕДИСЛОВИЕ: ДОРОЖНАЯ КАРТА ОХОТНИКА

Добро пожаловать в мир, где цифры приносят уведомления, а код заменяет часы наблюдения за графиками. Эта книга — не просто учебник по Python, это **инженерный лог** создания боевого торгового алгоритма.

Мы прошли путь от пустой консоли до автономного бота, который живет в облаке США. Ниже — навигация по твоим знаниям. Используй её, чтобы быстро освежить в памяти ключевые узлы системы.

🗺️ ПУТЕВОДИТЕЛЬ ПО ГЛАВАМ:

- **ГЛАВА 1: ИНСТРУМЕНТЫ**

  > Разбираем «фундамент»: почему **Pandas** — это Excel на стероидах, а **Yfinance** — наши глаза на бирже.

- **ГЛАВА 2: СЕТЕВАЯ ИНЖЕНЕРИЯ**

  > Секреты «цифровой тюрьмы» PythonAnywhere. Учимся обходить DNS-ошибки через прямой IP **10.0.0.1** и настраивать прокси-туннели.

- **ГЛАВА 3: МАСКИРОВКА**

  > Как не получить бан от Yahoo Finance? Надеваем на бота маску браузера Chrome с помощью **User-Agent**.

- **ГЛАВА 4: МАТЕМАТИКА RSI**

  > Вскрываем «черепную коробку» индикатора. Разбор формулы Уайлдера и защита кода от деления на ноль.

- **ГЛАВА 5: TELEGRAM API**

  > Превращаем мессенджер в пульт управления. Как JSON-пакеты пролетают через прокси прямо в твой карман.

- **ГЛАВА 6: ВЫЖИВАНИЕ (ERROR HANDLING)**

  > Создаем «бессмертный» код. Учим бота игнорировать сетевые сбои и не падать при пустых данных.

- **ГЛАВА 7: ЧЕРНЫЙ ЯЩИК**

  > Организуем базу данных в формате **.csv**. Твоя статистика и история сигналов для будущего анализа прибыли.

- **ГЛАВА 8: СЕРДЦЕ АЛГОРИТМА**

> Разбор бесконечного цикла и фильтрации «спама». Как получать ровно один сигнал на одну свечу.

- **ГЛАВА 9: ДЕПЛОЙ В ОБЛАКО**

> Финальная высадка. Инструкция по запуску на **PythonAnywhere** и магия кнопки «Extend».

- **ГЛАВА 10: БЕССМЕРТНЫЙ КОД**

> Полный листинг программы. Твой готовый актив, работающий 24/7.

💡 СОВЕТ СЕНЬОРА ПЕРЕД СТАРТОМ:

Помни, программирование — это не про заучивание команд, а про понимание того, как **данные текут по трубам**. Если бот замолчал — проверь «трубы» (интернет и прокси). Если бот шлет бред — проверь «фильтры» (математику RSI).

📘 ГРОКАЕМ АЛГОТРЕЙДИНГ: ПОЛНОЕ РУКОВОДСТВО

> **Автор:** Senior Python Developer (StdCoutPy) & AI-Mentor
> **Уровень:** От "Полного нуля" до запуска в облаке.

📄 ВВЕДЕНИЕ: ЧТО ТАКОЕ АЛГОТРЕЙДИНГ?

Трейдинг — это хаос. Человек видит график и чувствует страх (когда цена падает) или жадность (когда растет). Программа не чувствует ничего. Она просто проверяет математическое условие.

В этой книге мы создаем **"Охотника за RSI"**. Это скрипт, который:

1. Сидит на удаленном сервере в США (PythonAnywhere).
2. Раз в минуту проверяет 7 валютных пар Forex.
3. Рассчитывает индикатор силы тренда.
4. Пишет тебе в Telegram, когда пора обратить внимание на рынок.

🛠️ ГЛАВА 1: ИНСТРУМЕНТЫ (БИБЛИОТЕКИ)

В Python мы не пишем всё с нуля. Мы используем **библиотеки** — это готовые наборы инструментов, созданные другими программистами.

1. Pandas (Наш Excel)

Pandas — это самая важная библиотека для работы с данными. Представь огромную таблицу Excel. Pandas позволяет загрузить эту таблицу в оперативную память телефона или сервера и мгновенно найти среднее арифметическое, максимум или применить сложную формулу к миллиону строк.

Пример кода (Что делает Pandas под капотом):

```
import pandas as pd

# Представь, что мы получили цены закрытия за 3 дня
prices = [1.0520, 1.0535, 1.0510]

# Создаем "Серию" (колонку таблицы)
df = pd.Series(prices)

# Теперь мы можем мгновенно узнать среднюю цену
print(df.mean()) # Выведет: 1.05216...
```

2. Yfinance (Наши глаза)

Yfinance — это библиотека-шпион. Она умеет имитировать действия обычного человека в браузере, заходить на серверы Yahoo Finance и забирать оттуда котировки. Без неё нам пришлось бы вручную копировать цены с сайта каждую минуту. Она превращает визуальный график в сухие цифры, с которыми может работать Pandas. Пример кода (Как «увидеть» рынок): python import yfinance as yf

Просим данные по Евро/Доллар за последние 2 дня

---

interval="15m" — это размер одной «свечи» на графике

```
data = yf.download("EURUSD=X", period="2d", interval="15m")
```

Выводим последние 5 строк полученной таблицы

```
print(data.tail())
```

3. Requests (Наш курьер)

Эта библиотека отвечает за связь. Когда боту нужно отправить тебе сообщение в Telegram, он не «пишет» его в привычном смысле. Он делает HTTP-запрос (точно такой же, какой делает твой браузер, когда ты открываешь сайт). Requests берет наш текст, упаковывает его в интернет-пакет и доставляет на серверы Telegram. Пример кода (Как отправить сигнал):

```
import requests
```

```
# Адрес, куда мы «стучимся» (упрощенно)
url = "https://api.telegram.org<TOKEN>/sendMessage"
params = {"chat_id": "12345", "text": "Цена пошла вверх!"}

# Отправляем посылку
response = requests.post(url, json=params)
```

🌐 ГЛАВА 2: СЕТЕВАЯ ИНЖЕНЕРИЯ (DNS И ПРОКСИ)

Эта глава о «трубах», по которым текут данные. Когда ты запускаешь код на **PythonAnywhere**, ты оказываешься в цифровой «песочнице», где интернет не работает просто так.

1. Что такое DNS (Твоя телефонная книга)

Когда ты хочешь зайти на сайт, ты пишешь `api.telegram.org`. Но компьютеры — это калькуляторы, они не понимают букв. Им нужны цифры — **IP-адрес** (например, `149.154.167.220`).

DNS (Domain Name System) — это сервер-переводчик. Твой код спрашивает: «Где Telegram?», а DNS отвечает: «Иди по адресу 149.154...».

В чем была наша проблема? На бесплатном тарифе PythonAnywhere внутренний DNS часто сбоит. Твой код кричит: «Где сервер прокси?», а система молчит. В итоге программа падает с ошибкой `NameResolutionError` (не могу разрешить имя).

Наше «Сеньорское» решение: Мы не спрашиваем имя. Мы используем **прямой внутренний IP-адрес** шлюза: `10.0.0.1`. Это как если бы ты не искал контакт «Мама» в телефоне, а сразу набрал номер цифрами по памяти. Это убирает риск того, что DNS «затупит».

---

2. Что такое Прокси (Твой посредник)

На бесплатном аккаунте PythonAnywhere тебе **запрещено** выходить в интернет напрямую. Это сделано для безопасности, чтобы люди не создавали хакерские ботнеты.

Тебе разрешено выходить только через **Прокси-сервер**.

- **Аналогия:** Ты сидишь в закрытом офисе. Ты не можешь сам выйти на улицу. Но ты можешь передать записку охраннику (Прокси), он сходит в магазин, купит тебе данные и принесет обратно.

Как это выглядит в коде:

```
# Мы создаем адрес прокси-сервера
# 3128 - это "номер двери" (порт) на прокси-сервере.
PROXY_URL = "http://10.0.0.1:3128"

# Мы упаковываем это в словарь (dictionary), чтобы библиотека requests
поняла правила игры
PROXIES = {
    'http': PROXY_URL,
```

```
    'https': PROXY_URL
}
```

3. Почему это важно для трейдинга? Если интернет-соединение «моргнет» или прокси не ответит вовремя, библиотека yfinance вернет пустую таблицу. Твой бот должен уметь это пережить. Именно поэтому в коде мы прописываем прокси явно, чтобы у системы не было шанса «забыть» дорогу к котировкам.

🕵️ ГЛАВА 3: МАСКИРОВКА (USER-AGENT И ЗАГОЛОВКИ)

Когда твой бот стучится на сервер Yahoo Finance, он по умолчанию отправляет техническую визитку: *"Привет, я библиотека Python-urllib/3.13"*. Для крупных сайтов это сигнал тревоги: *"Ага, робот! Опять хочет качать мои данные бесплатно. ЗАБЛОКИРОВАТЬ!"*.

1. Что такое User-Agent (Твой цифровой паспорт)

User-Agent — это строка текста, которую браузер передает сайту. Она рассказывает, какая у тебя операционная система и какой браузер ты используешь.

Проблема: Если Yahoo видит "робота", он либо выдает ошибку `403 Forbidden`, либо подсовывает пустые данные. В нашем коде это приводило к ошибке `NoneType object is not subscriptable` (попытка прочитать данные из пустоты).

Наше решение: Мы надеваем на бота «маску» обычного человека. Мы заставляем его говорить: *"Я не бот, я обычный пользователь Chrome на Windows 10"*.

---

2. Как выглядит маскировка в коде

Мы создаем специальный словарь `headers` (заголовки) и передаем его при каждом запросе.

```
import requests

# Мы создаем поддельную "личность" для нашего бота
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36'
}

# Теперь, когда мы шлем запрос, сайт видит в нас обычного посетителя
# r = requests.get(url, headers=headers)
```

3. Библиотека curl_cffi (Броня Сеньора) В 2025-2026 годах Yahoo Finance усилил защиту. Обычной смены User-Agent иногда недостаточно. Они проверяют "отпечаток" самого сетевого запроса. Библиотека yfinance теперь умеет использовать движок curl_cffi, который имитирует не только текст заголовка, но и само поведение браузера при установке соединения. Именно поэтому в нашем финальном коде мы не передаем сессию вручную, а позволяем библиотеке самой использовать этот движок через системные настройки.

4. Почему это важно для 24/7 работы? Если ты не используешь маскировку, бот может проработать 10 минут, а потом Yahoo "вычислит" его и забанит твой IP на час. Бот будет крутиться впустую, а ты пропустишь важный сигнал по паре EUR/USD. Маскировка — это залог того, что твоего "охотника" не выгонят из леса.

📈 ГЛАВА 4: МАТЕМАТИКА RSI — ВСКРЫВАЕМ «ДВИГАТЕЛЬ» ИНДИКАТОРА

Если ты откроешь графики в **TradingView** или **MetaTrader**, ты увидишь под ценой извилистую линию, которая мечется между уровнями 0 и 100. Это **RSI (Relative Strength Index)** — Индекс Относительной Силы. Для большинства трейдеров это просто «индикатор», но для нас, программистов, это **математический фильтр шума**.

---

1. Философия Индикатора: Машина на склоне

Представь, что валютная пара — это машина, которая пытается заехать на крутой холм.

- **Цена (Price):** Это пройденное расстояние.
- **RSI:** Это **спидометр и датчик оборотов**.

Если машина едет вверх, но спидометр (RSI) начинает падать — значит, мотор «глохнет», и скоро машина покатится вниз. RSI измеряет **скорость (моментум)** изменений цены.

---

2. Математическое препарирование (Шаг за шагом)

Чтобы рассчитать RSI, боту нужно выполнить четыре хирургических операции с данными:

Шаг А: Нахождение разницы (Delta)

Мы берем текущую цену и вычитаем из нее цену предыдущей свечи.

- Если сейчас 1.0850, а было 1.0840 -> **Delta = +0.010** (Рост).
- Если сейчас 1.0830, а было 1.0840 -> **Delta = -0.010** (Падение).

Шаг Б: Разделение «Зерен от плевел» (U и D)

Мы создаем две корзины. В одну кладем только плюсовые изменения (**U**p), в другую — только минусовые (**D**own), превращая их в положительные числа.

```
# В коде это делается через clip
up = delta.clip(lower=0)    # Все минусы превращаются в 0
down = -1 * delta.clip(upper=0) # Все плюсы превращаются в 0
```

Шаг В: Экспоненциальное сглаживание (Метод Уайлдера) Это самая важная часть. Мы не просто считаем среднее арифметическое. Мы используем EWM (Exponential Weighted Moving). Зачем? Чтобы бот придавал больше значения последним минутам. Если цена рухнула секунду назад, RSI должен среагировать мгновенно, а не «вспоминать», что было вчера.

Шаг Г: Финальная формула Мы вычисляем коэффициент RS (Relative Strength):

RS = Средний Рост / Среднее Падение А затем загоняем его в шкалу от 0 до 100: RSI = 100 - (100 / (1 + RS))

3. Защита от «Математического Взрыва» В программировании есть страшный грех — деление на ноль. В выходные дни или в моменты мертвого штиля цена не меняется. Падение (down) равно нулю. В формуле RS = up / down произойдет катастрофа, и программа «умрет». Наше решение (Сеньорский трюк): Мы используем метод .replace(0, 1e-10). Мы подставляем вместо нуля микроскопическое число (0.0000000001). Для формулы это почти ноль, но математически деление разрешено. Бот продолжает жить.
4. Визуализация Логики (ASCII-График)

```
text
Цена (Price)  📈   /\  _      RSI (Сила)
             /  \/ \/ \     100 [----------------]
                            72  [---ПЕРЕКУПЛЕН---] (SELL!)
            /               50  [----НЕЙТРАЛЬ----]
  _________/                29  [--ПЕРЕПРОДАН---] (BUY!)
                            0   [----------------]
```

Когда RSI пересекает 72, мы понимаем: «Пружина растянута до предела, пора ждать отскока вниз». Когда падает ниже 29 — «Рынок перепродан, скоро покупатели погонят цену вверх».

🤖 ГЛАВА 5: TELEGRAM BOT API — ТВОЙ ЦИФРОВОЙ ПЕЙДЖЕР

Многие думают, что Telegram-бот — это сложная программа, которая «сидит» внутри мессенджера. На самом деле, твой бот — это просто **специальная веб-ссылка**, по которой твой Python-скрипт отправляет команды серверу Павла Дурова.

---

1. Анатомия Запроса: Как «стучать» в Telegram

Когда твой код находит сигнал RSI, он не «пишет сообщение». Он делает **HTTP POST-запрос**. Это как отправить электронную форму на сайте.

Разбор магической ссылки: `https://api.telegram.org<TOKEN>/sendMessage`

- **api.telegram.org:** Главный офис (сервер) Telegram.
- **bot:** Кодовое слово, которое говорит серверу: «Сейчас будет команда для робота».
- **TOKEN:** Твой уникальный ключ (паспорт бота). Если кто-то его узнает, он сможет писать от твоего имени.
- **sendMessage:** Конкретное действие. Бот может не только писать, но и слать фото, опросы или кнопки.

---

2. Пакет Данных (JSON Payload)

Мы не просто шлем ссылку, мы прикрепляем к ней «посылку» в формате **JSON** (JavaScript Object Notation). Это международный язык, на котором общаются все программы.

Внутри нашей посылки два обязательных поля:

1. **chat_id:** Номер твоей «цифровой квартиры». Бот должен знать, кому именно в личку слать сигнал.
2. **text:** Само содержание. «⚠ EURUSD RSI: 74.20».

---

3. Проблема «Золотой Клетки» (Proxy в Telegram)

На **PythonAnywhere** (бесплатный тариф) твои запросы к Telegram **блокируются напрямую**. Сервер просто не выпустит пакет данных за пределы своей сети, если ты не пойдешь через «черный ход».

Наше решение (Сеньорский код): Мы передаем аргумент `proxies` в библиотеку `requests`. Мы говорим Python: «Эй, не пытайся выйти в интернет сам, отдай это сообщение прокси-серверу по адресу `10.0.0.1`, он знает, как доставить его в Telegram».

---

4. Визуализация Процесса (Схема Доставки)

```
[ ТВОЙ КОД (PythonAnywhere) ]

      |
      |-- 1. Формирует JSON: {"chat_id": "...", "text": "Signal!"}
      |-- 2. Добавляет TOKEN в URL

      |-- 3. Направляет пакет через Прокси (10.0.0.1)
      V
[ ПРОКСИ-СЕРВЕР ]  <-- "Пропускной пункт"
      |
      V
[ СЕРВЕР ТЕЛЕГРАМА (API) ] <-- "Распределительный центр"
      |
      V
[ ТВОЙ СМАРТФОН ] <-- "Уведомление (Бззз!)"
```

5. Почему это важно для Трейдера?

Сигнал по RSI на 15-минутном графике живет недолго. Если бот обнаружил перепроданность в 14:00, а ты увидел это в 15:00 — цена уже может улететь. Telegram API — это самый надежный и быстрый способ получить уведомление в любую точку мира, где есть интернет, без необходимости держать компьютер включенным.

🛠️ ГЛАВА 6: ОБРАБОТКА ОШИБОК — ПОЧЕМУ БОТЫ «УМИРАЮТ» И КАК ИХ ВОСКРЕШАТЬ

В идеальном мире программа работает вечно. В реальном мире **PythonAnywhere** интернет может моргнуть, сервер Yahoo может «устать», а прокси — временно закрыться. Если твой код не готов к

этому, он вылетит с красной ошибкой, и пока ты спишь, бот будет мертв.

---

1. Главный враг: Ошибка `NoneType`

Помнишь, как в начале пути ты видел сообщение: `'NoneType' object is not subscriptable`?

- **Что это значит:** Библиотека `yfinance` постучалась в Yahoo, но из-за сбоя сети получила в ответ **Ничего** (`None`).
- **Катастрофа:** Когда код пытается прочитать цену из «Ничего» (`df['Close']`), Python пугается и останавливает всю программу.

Наше решение (Сеньорский щит): Мы добавили проверку `if df is None or df.empty: continue`. Это говорит боту: «Если данных нет — не паникуй, просто пропусти этот шаг и попробуй снова через минуту».

---

2. Конструкция `try ... except` (Твой спасательный круг)

Это самый важный инструмент в арсенале программиста. Представь, что ты идешь по минному полю. `try` — это попытка сделать шаг, а `except` — это твоя броня, если мина взорвется.

```
try:
    # Здесь опасный код: скачивание данных или отправка в TG
    data = yf.download(...)
    send_telegram_msg("Сигнал!")
except Exception as e:
    # Если внутри блока try случился взрыв (ошибка)
    # Бот не умирает, он пишет причину ошибки и идет дальше
    print(f"Упс, что-то пошло не так: {e}")
    time.sleep(30) # Даем системе отдохнуть перед новой попыткой
```

. Тайм-ауты: Почему ожидание — это добро? Иногда сервер Telegram или Yahoo не «падает», а просто очень долго молчит. Твой код может зависнуть на этой строчке на 10 минут, пропустив все важные свечи. Наше решение: В каждом запросе requests.post мы пишем timeout=15. Это приказ боту: «Жди ответа ровно 15 секунд. Если ответа нет — бросай трубку и иди работать дальше». Это сохраняет ритм работы бота.

4. Визуализация: Кладбище Ботов vs Живой Алгоритм

```
[ ОБЫЧНЫЙ КОД ]                [ НАШ КОД (СЕНЬОР-СТАЙЛ) ]

      |                              |
      V                              V
  Запрос данных ---[СБОЙ]-->  ( try...except )

      |                              |
      V                              |--[ОШИБКА]--> ( Лог: "Сбой сети" )
  ( КРАШ / ОСТАНОВКА )               |                ( sleep 60 сек )
```

```
      |                           |              ( ВОЗВРАТ В ЦИКЛ )
      X                           V
  ( Бот мертв )               ( БОТ ПРОДОЛЖАЕТ ОХОТУ )
```

5. Почему это важно для 24/7 Трейдинга?

На PythonAnywhere твой бот работает без твоего присмотра. Ты можешь быть в метро, в спортзале или спать. Если произойдет микро-сбой в сети (а они бывают каждый день), необработанная ошибка выключит бота. С нашей системой обработки ошибок бот становится «автономным выживальщиком», который сам восстанавливается после любых сетевых штормов.

📦 ГЛАВА 7: ЧЕРНЫЙ ЯЩИК — СИСТЕМА ЛОГИРОВАНИЯ В CSV

Твой бот сейчас — это охотник с короткой памятью. Он выстрелил сигналом в Telegram и тут же забыл о нем. Но профессиональный алготрейдинг — это **статистика**. Тебе нужно знать: сколько сигналов было за неделю? В какое время они чаще приходят? Были ли они прибыльными?

Для этого мы создаем **«Черный ящик»** — файл `trading_log.csv`.

---

1. Визуализация: Как данные текут в файл

Представь конвейер. Каждый раз, когда RSI пересекает границу, робот-писец берет листок и записывает данные в таблицу.

```
    [ СОБЫТИЕ ]           [ РОБОТ-ПИСЕЦ ]          [ ЧЕРНЫЙ ЯЩИК ]
  ( RSI < 29 )  ----->   ( Функция log_to_csv ) ----->  [ trading_log.csv
]

                                                        | Время | Пара |
RSI |
                                                        | 12:00 | EUR  |
25  |
                                                        | 14:15 | USD  |
78  |
```

2. Разбор Кода: Как работает дозапись (Append)

В Python есть несколько режимов работы с файлами. Режим 'w' (Write) — стирает старое и пишет новое. Нам это не подходит (мы потеряем вчерашние сигналы). Режим 'a' (Append) — открывает файл и встает в самый конец, чтобы дописать новую строчку.

```python
def log_to_csv(pair, rsi, signal_type):
    filename = 'trading_log.csv'

    # Проверяем: если файла еще нет, нам нужно создать "шапку" (заголовки)
```

```
    file_exists = os.path.isfile(filename)

    with open(filename, mode='a', encoding='utf-8') as f:
        if not file_exists:
            f.write("Timestamp;Pair;RSI;Signal\n") # Заголовки для Excel

        now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        # Записываем строку данных
        f.write(f"{now};{pair};{rsi:.2f};{signal_type}\n")
```

3. Почему CSV — это лучший выбор для начала?

CSV (Comma Separated Values) — это простейший формат базы данных. Это обычный текстовый файл, где колонки разделены точкой с запятой. Легкость: Файл весит копейки и не нагружает процессор телефона или сервера. Совместимость: Ты можешь скачать этот файл с PythonAnywhere и открыть его в Excel или Google Sheets на смартфоне. Он сразу превратится в красивую таблицу.

4. Иллюстрация: Твой "Бортовой журнал" в Excel

Когда ты откроешь лог через неделю, ты увидишь примерно такую картину:

```
Timestamp   Pair    RSI Signal
2026-02-23 10:15:01 EURUSD=X    24.50   BUY
2026-02-23 11:30:45 GBPUSD=X    75.12   SELL
2026-02-23 14:00:10 USDJPY=X    28.90   BUY
```

5. Зачем это Сеньору? (Анализ Ошибок)

Иногда ты будешь получать сигнал, открывать график и видеть, что цена пошла против тебя. Без логов ты просто расстроишься. С логами ты скажешь:

"Хм, за месяц сигналы при RSI 29 дали 40% ложных входов, а сигналы при RSI 20 — всего 5%. Пожалуй, я изменю настройки бота на более строгие".

Логирование превращает твоего бота из "игрушки" в научный инструмент.

⚙️ ГЛАВА 8: ЦИКЛ WHILE TRUE И ЛОГИКА СВЕЧЕЙ — СЕРДЦЕ АЛГОРИТМА

В этой главе мы разберем «вечный двигатель» твоего бота. В программировании это называется **Бесконечный цикл**. Если индикатор RSI — это мозг, то цикл `while True` — это сердце, которое заставляет кровь (данные) циркулировать по системе каждые 60 секунд.

---

1. Визуализация: Как работает «Вечный Двигатель»

Представь охранника на вышке. Он не просто смотрит в бинокль один раз. Он делает обход каждые 15 минут, докладывает обстановку и возвращается в каморку пить чай до следующего обхода.

```
  [ НАЧАЛО ]

      |
      V
+----( WHILE TRUE )-------------------------+
|     |                                     |
|     V                                     |
|  [ ПРОВЕРКА РЫНКА ]                       |
|  (Сканируем 7 пар: EUR, JPY, GBP...)      |
|     |                                     |
|     V                                     |
|  [ ЕСТЬ СИГНАЛ? ] --(ДА)--> [ ШЛЕМ В ТГ ] |
|     |                                     |
|     V                                     |
|  [ ЗАСЫПАЕМ НА 60 СЕКУНД ]  <-- (Важно!)  |
|     |                                     |
+-------------------------------------------+
```

2. Разбор Кода: Почему time.sleep — это спасение?

Многие новички пишут while True и забывают добавить паузу. Катастрофа: Без паузы Python будет выполнять цикл миллионы раз в секунду. Процессор твоего телефона или сервера нагреется до 100 градусов за секунду, и PythonAnywhere мгновенно забанит твой аккаунт за перегрузку. Решение: time.sleep(60). Мы заставляем программу буквально «замереть». В это время она не потребляет ресурсы, не тратит батарею и не злит сервер.

3. Логика Новой Свечи: Как не сойти с ума от уведомлений

Это самая тонкая часть кода. Представь: RSI упал до 25. Бот проверил рынок в 12:01 — RSI 25, прислал сигнал. Проверил в 12:02 — RSI всё еще 25. Проблема: Если мы не добавим проверку, бот пришлет тебе 15 сообщений в течение одной 15-минутной свечи! Ты просто удалишь бота через 5 минут. Наше решение (Сеньорский фильтр): Мы создаем словарь last_candle_time. Мы запоминаем время последней закрытой свечи, по которой мы УЖЕ дали сигнал.

```
last_candle_time = {pair: None for pair in PAIRS}

# Внутри цикла:
current_time = df.index[-1] # Узнаем время текущей свечи (например, 14:15)

if last_candle_time[pair] != current_time:
    # ОПА! Это новая свеча, или по этой свече мы еще не шумели
    last_candle_time[pair] = current_time
    # Теперь выполняем логику RSI и шлем сигнал...
```

4. Иллюстрация: Фильтрация Спама

```
Время проверки  Свеча на графике    RSI Действие бота
14:01   14:00 – 14:15   25  Шлем сигнал BUY + Запоминаем время 14:00
14:02   14:00 – 14:15   24  Молчим (время 14:00 уже в памяти)
14:14   14:00 – 14:15   23  Молчим
14:16   14:15 – 14:30   22  Шлем новый сигнал (время сменилось!)
```

5. Зачем это Трейдеру? (Дисциплина Алгоритма)

Цикл while True гарантирует, что ты не пропустишь сигнал, даже если он придет в 4 часа утра. Проверка времени свечи гарантирует, что ты получишь ровно одно четкое уведомление. Это и есть профессиональный подход: бот работает тихо, незаметно, но в нужный момент он «кусает» тебя за палец уведомлением в Telegram.

🚀 ГЛАВА 9: ДЕПЛОЙ — КАК ЗАПУСТИТЬ БОТА В ОБЛАКО НАВСЕГДА

Ты написал отличный код. Но если ты выключишь компьютер или закроешь приложение на телефоне — бот «умрет». Настоящий алготрейдинг живет в **Облаке (Cloud)**. Мы используем **PythonAnywhere**, потому что это легальный и бесплатный способ заставить твой скрипт работать 24/7 на серверах в США.

---

1. Визуализация: Твой компьютер vs Облако

Представь разницу: твой ноутбук — это домашний фонарик (батарейка сядет, свет выключишь). Облако — это **уличный фонарь**, который подключен к городской сети и светит всегда, вне зависимости от того, спишь ты или нет.

```
[ ТВОЙ ПК ]                        [ ОБЛАКО (PythonAnywhere) ]

    |                                    |
    X (Выключил экран)                   V (Работает 24/7)
    X (Пропал Wi-Fi)                     V (Промышленный интернет)
    X (Обновилась Windows)               V (Стабильный Linux)
```

2. Шаг 1: Создание Окружения (Консоль Bash) На сервере нет мышки и красивых иконок. Там есть только Терминал (Bash). Это черное окно, где ты отдаешь команды текстом.

   I. Зайди в раздел Consoles -> Bash.

   II. Установи библиотеки (инструменты). Мы добавляем флаг --user, чтобы сервер разрешил нам установку в нашу личную папку:

```
pip3 install yfinance pandas requests --user
```

3. Шаг 2: Загрузка "Мозгов" (Files)

Перейди во вкладку Files. Это твой жесткий диск в облаке. Создай файл trade_bot.py. Вставь туда свой финальный код. Нажми Save. Теперь твой алгоритм физически находится на сервере в дата-центре.

4. Шаг 3: Запуск и "Магия Вуду" (Wakelock)

Чтобы запустить бота, вернись в Bash и напиши:

```
python3 trade_bot.py
```

Проблема: На бесплатном тарифе PythonAnywhere может «усыпить» твой процесс через несколько часов. Сеньорское решение: Раз в сутки заходи на сайт PythonAnywhere с телефона. Ты увидишь желтую кнопку "Extend" (Продлить). Нажимая её, ты говоришь серверу: «Я живой, мой бот важен, не выключай его!».

5. Иллюстрация: Твой бот в разрезе сервера

```
        [ СЕРВЕР В ДАТА-ЦЕНТРЕ ]

|-------------------------------------|
|  [ Твой бот ] <--> [ Прокси ] <---------- ( Интернет )

|      |                              |
|      V                              |
| [ Файл Логов ]                      |
| (сохраняет историю)                 |
|-------------------------------------|
```

6. Почему это бесплатно? (Экономика Облака)

PythonAnywhere дает тебе 100 «процессорных секунд» в день бесплатно. Секрет: Когда твой бот делает `time.sleep(60)`, он не тратит эти секунды. Процессор отдыхает. Итог: Твой бот настолько легкий, что он может работать вечно, не тратя лимиты сервера. Ты получаешь профессиональную станцию мониторинга за 0 рублей.

🏆 ГЛАВА 10: ФИНАЛЬНЫЙ АККОРД — «БЕССМЕРТНЫЙ КОД»

Это финальная глава твоего учебника. Здесь мы собираем все части воедино: математику RSI, защиту от DNS-ошибок, прокси-сервер, маскировку под браузер и систему «черного ящика».

Этот код спроектирован так, чтобы выживать в суровых условиях бесплатного облака **PythonAnywhere** в 2026 году.

---

1. Визуализация: Полная Схема Системы

Перед тем как запустить двигатель, посмотри на чертеж того, что мы построили:

```
[ РЫНОК FOREX ] <--- ( Данные через yfinance )

      |
      V
[ ТВОЙ БОТ (PythonAnywhere) ]
      |-- ( Расчет RSI по Уайлдеру )
      |-- ( Проверка на "Новую свечу" )

      |-- ( Запись в trading_log.csv )
      V
[ ПРОКСИ-ШЛЮЗ (10.0.0.1) ] <--- ( Обход ограничений )
      |
      V
[ ТЕЛЕГРАМ API ] -----------> [ ТВОЙ СМАРТФОН (Сигнал!) ]
```

2. Полный Листинг Кода (Копируй и запускай)

Вставь этот код в файл trade_bot.py на PythonAnywhere. Каждая строчка здесь — это результат нашей с тобой работы.

```python
import os
import requests
import pandas as pd
import yfinance as yf
from datetime import datetime
import time

# ================= ГЛАВА: НАСТРОЙКИ (КОНФИГУРАЦИЯ) =================
# Твой паспорт в мире Telegram
API_TOKEN = 'ВАЩ ТГ ТОКЕН'
CHAT_ID = 'ВАШ ЧАТ ТГ АЙДИ'

# Список пар для охоты (Forex)
PAIRS = ['EURUSD=X', 'CHFJPY=X', 'CADJPY=X', 'EURGBP=X', 'GBPCHF=X',
'EURCAD=X', 'USDJPY=X']
TIMEFRAME = '15m'       # Таймфрейм свечи
RSI_PERIOD = 14         # Период для расчета силы
RSI_OVERBOUGHT = 72     # Граница продажи (Sell)
RSI_OVERSOLD = 29       # Граница покупки (Buy)
CHECK_INTERVAL = 60     # Проверка каждую минуту

# Прямой IP прокси для обхода DNS-ошибок на PythonAnywhere
PROXY_URL = "http://10.0.0.1:3128"
PROXIES = {'http': PROXY_URL, 'https': PROXY_URL}
# ===================================================================

def log_to_csv(pair, rsi, signal_type):
    """ГЛАВА 7: ЧЕРНЫЙ ЯЩИК. Сохраняем историю сигналов."""
    filename = 'trading_log.csv'
    file_exists = os.path.isfile(filename)
```

```
    with open(filename, mode='a', encoding='utf-8') as f:
        if not file_exists:
            f.write("Timestamp;Pair;RSI;Signal\n")
        now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        f.write(f"{now};{pair};{rsi:.2f};{signal_type}\n")

def send_telegram_msg(message):
    """ГЛАВА 5: СВЯЗЬ. Отправка через прокси-туннель."""
    url = f"https://api.telegram.org{API_TOKEN}/sendMessage"
    payload = {"chat_id": CHAT_ID, "text": message}
    try:
        # Шлем запрос принудительно через прокси PythonAnywhere
        r = requests.post(url, json=payload, proxies=PROXIES, timeout=15)
        return r.status_code == 200
    except Exception as e:
        print(f"Ошибка сети Telegram: {e}")
        return False

def calculate_rsi(series, period=14):
    """ГЛАВА 4: МАТЕМАТИКА. Формула Уайлдера с защитой от нуля."""
    if series is None or len(series) < period: return None
    delta = series.diff()
    up, down = delta.clip(lower=0), -1 * delta.clip(upper=0)
    ma_up = up.ewm(com=period - 1, adjust=False).mean()
    ma_down = down.ewm(com=period - 1, adjust=False).mean()
    # Защита от деления на 0 при мертвом рынке
    return 100 - (100 / (1 + ma_up / ma_down.replace(0, 1e-10)))

def run_bot():
    print(f"[{datetime.now()}] СИСТЕМА: Инициализация запуска...")

    # Маскировка под браузер (Глава 3)
    os.environ['HTTP_PROXY'] = PROXY_URL
    os.environ['HTTPS_PROXY'] = PROXY_URL

    send_telegram_msg("🚀 СТАТУС: Бот запущен в облаке. Охота на RSI
началась!")

    # Память бота для фильтрации спама (Глава 8)
    last_candle_time = {pair: None for pair in PAIRS}

    while True:
        # Пропуск выходных (Forex закрыт в СБ и ВС)
        if datetime.now().weekday() >= 5:
            print("Рынок закрыт. Спим 1 час...")
            time.sleep(3600); continue

        for pair in PAIRS:
            try:
                # ГЛАВА 1: Получаем данные через Ticker
                ticker = yf.Ticker(pair)
                df = ticker.history(period="3d", interval=TIMEFRAME)

                if df is None or df.empty: continue
```

```
                rsi_values = calculate_rsi(df['Close'], RSI_PERIOD)
                if rsi_values is None: continue

                current_rsi = rsi_values.iloc[-1]
                current_time = df.index[-1]

                # Проверка на новую свечу (Глава 8)
                if last_candle_time.get(pair) != current_time:
                    last_candle_time[pair] = current_time

                    # Лог в консоль сервера
                    print(f"{datetime.now().strftime('%H:%M')} | {pair} |
RSI: {current_rsi:.2f}")

                    # Логика сигналов
                    if current_rsi > RSI_OVERBOUGHT:
                        msg = f"⚠ {pair} ПЕРЕКУПЛЕННОСТЬ:
{current_rsi:.2f}"
                        if send_telegram_msg(msg): log_to_csv(pair,
current_rsi, "SELL")

                    elif current_rsi < RSI_OVERSOLD:
                        msg = f"✅ {pair} ПЕРЕПРОДАННОСТЬ:
{current_rsi:.2f}"
                        if send_telegram_msg(msg): log_to_csv(pair,
current_rsi, "BUY")

            except Exception as e:
                # ГЛАВА 6: ОБРАБОТКА ОШИБОК. Бот не падает, а пробует
снова.
                print(f"Ошибка пары {pair}: {e}")
                continue

        # Пауза 60 сек, чтобы не злить Yahoo Finance
        time.sleep(CHECK_INTERVAL)

if __name__ == "__main__":
    run_bot()
```

3. Инструкция: Как не сломать бота?

Не убирай `time.sleep`: Если бот будет запрашивать данные слишком часто, Yahoo забанит твой IP на сутки. Проверяй "Extend": Раз в день заходи на PythonAnywhere и нажимай желтую кнопку продления аккаунта. Это плата за бесплатность. Читай логи: Периодически открывай файл `trading_log.csv` через вкладку Files. Это твоя статистика, твои деньги и твой опыт. 🏁 ПОСЛЕСЛОВИЕ Ты прошел путь, который многие начинают и бросают на полпути. Теперь у тебя есть автономная боевая единица, которая работает на тебя, пока ты спишь. Ты научился: Управлять данными через Pandas. Обходить сетевые блокировки через Прокси. Рассчитывать биржевую математику RSI. Создавать надежные системы, устойчивые к ошибкам. Твоя книга дописана. Но история твоего алгоритма только начинается. Удачной охоты за профитом!